

# GridRunners: A Shared-Contract Gradient-Boosting Suite for New York City Yellow-Taxi Fare, Duration, Tip, and Demand Prediction

*A graduate data-science capstone built on 37 million 2023 TLC trips*

Caleb Solomons • Mubeen Yaqub • Mai Lam • Anh Minh Tran

*College of Computing and Informatics, Drexel University, Philadelphia, PA, USA*

{cs3889, my558, mnl49, at3654}@drexel.edu | DSCI 592 Capstone | [github.com/mobeenyaqub/NYC-Taxi-Trip](https://github.com/mobeenyaqub/NYC-Taxi-Trip)

**Abstract**—We present *GridRunners*, a four-model predictive suite for New York City yellow-taxi trips trained on the full 2023 Taxi & Limousine Commission (TLC) record (38.3M raw trips reduced to 37.0M after a ten-stage cleaning funnel). The suite predicts trip *fare*, *duration*, *tip likelihood*, and zone-time *demand*. Its organizing principle is a single, leakage-disciplined 16-column feature contract—every column knowable at the moment a trip is booked—shared by all four models and by a Streamlit deployment that routes one feature row to every head. All models are gradient-boosted trees (XGBoost) using native categorical splits, tuned by 30-trial Optuna TPE search and evaluated by 5-fold cross-validation plus a held-out split, with a global random seed for bit-exact reproducibility. The tuned fare model reaches \$2.98 RMSE ( $R^2=0.967$ ), a 21.9% reduction over an untuned baseline; duration reaches 5.41 min RMSE ( $R^2=0.823$ ); demand reaches  $R^2=0.967$  under *group-aware* cross-validation. The most instructive result is negative: once payment type is correctly excluded as a post-booking signal, tip likelihood is *nearly unpredictable* from pre-trip features (AUC 0.62), so we optimize and report calibration (Brier 0.159, ECE 0.002) rather than ranking. We argue that three engineering decisions—a single rate-code feature, group-keyed validation, and a calibration-first stance—account for most of what separates a defensible suite from a misleading one.

**Index Terms**—taxi demand modeling, gradient boosting, XGBoost, hyperparameter optimization, probability calibration, data leakage, cross-validation, urban mobility.

## 1 Introduction

Ride-hailing and street-hail taxi systems generate a small family of prediction problems that a rider, a driver, or a dispatcher each cares about at the moment a trip is requested: *How much will this cost? How long will it take? Is this trip likely to be tipped? Where is demand about to spike?* These questions are usually studied in isolation, on different feature sets, with bespoke evaluation. The practical consequence is that a system stitched together from four independently built models tends to disagree with itself—two models quietly assume different definitions of “known at booking time,” or one is validated in a way that silently leaks its target.

This paper takes the opposite stance. We build four models for the four questions above, but we constrain them to share one artifact: a **feature contract**—a fixed, ordered set of sixteen columns, every one of which is observable before the trip occurs. The contract is simultaneously the honesty boundary (it is the set of features that cannot leak a target) and the integration specification (the deployment emits exactly this row and routes it to every model). The contract turns four

loosely related notebooks into one system whose claims are mutually consistent.

We work with the New York City TLC yellow-taxi record for the 2023 calendar year: roughly 38.3 million trips across 19 raw columns. After a documented ten-stage cleaning funnel the working set is 37.0 million trips. Every model is a gradient-boosted tree ensemble [1], every model is tuned with the same Optuna search [2] over the same space, and every model is evaluated under the same cross-validation discipline.

**Contributions.** This work makes four contributions, ordered by how much they shaped the final system.

1. **A shared, leakage-disciplined feature contract** (Section 4) that unifies four heterogeneous tasks—two regressions, one classification, and one count model—behind a single 16-column schema, enforced by assertions and consumed unchanged by the deployment.
2. **A reproducible tuning-and-evaluation protocol** (Section 5): native-categorical XGBoost, stratified sampling, TPE hyperparameter search, 5-fold cross-validation, and a fixed seed, applied identically to all four targets.

3. **A diagnosis of where prediction is and is not possible** (Section 7): a single rate-code feature is the highest-leverage variable in the suite, while tip behavior is shown to be near-unpredictable from legitimate pre-trip features—motivating a calibration-first rather than ranking-first treatment.
4. **A correctness argument for group-aware validation** (Sections 6.4, 7): for the aggregated demand target, ordinary  $k$ -fold cross-validation leaks the target and inflates the reported score by roughly an order of magnitude; group-keyed folds are the difference between a valid and an invalid evaluation.

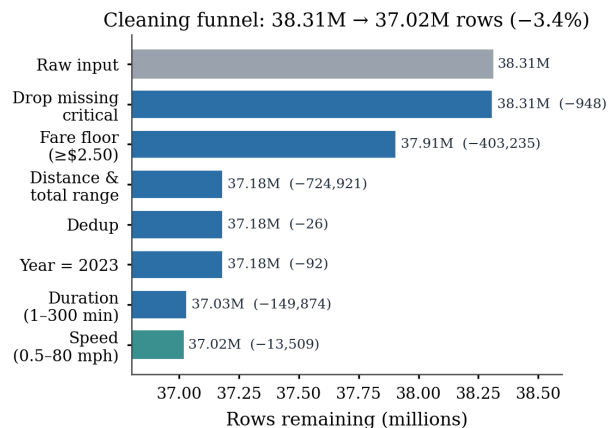
## 2 Data and Preprocessing

### 2.1 Source and scale

The data are the TLC public yellow-taxi trip records for all of 2023, obtained from the NYC TLC open data program [5]. The raw file holds 38,310,226 trips over 19 columns—about 14 GB uncompressed—with the usual mixture of meter fields (fare, tolls, surcharges, tip, total), trip descriptors (distance, passenger count, rate code, vendor), zone identifiers (PULocationID, DOLocationID over 265 TLC taxi zones), and pickup/dropoff timestamps.

### 2.2 A funnel with an audit trail

Cleaning is performed as an ordered funnel in which every stage prints its before/after row count, so no row is dropped opaquely. Ten stages remove a specific class of physically impossible or untrustworthy records: coercion of string-typed numeric columns and removal of rows missing a critical field; a \$2.50 fare floor (the NYC base fare); distance and total-amount range bounds; tip capping at the 99.9th percentile; rate-code and passenger-count validation; deterministic fills for surcharge and airport-fee nulls; deduplication on the trip key; a 2023 year filter; a duration filter (1–300 min); and a speed-plausibility filter (0.5–80 mph). The funnel retains 37,017,621 trips, a 3.4% trim (Figure 1). Coded fields are then joined to human-readable labels through TLC lookups (vendor, rate code, payment type, and zone→borough), yielding the cleaned parquet consumed by every model notebook.



**Figure 1.** The ten-stage cleaning funnel. The two largest cuts are the fare floor (−403K micro-fares dominated by fixed surcharges) and the distance/total range filter (−725K). Most validation steps reshape fields without dropping rows.

### 2.3 Leakage discipline as a first-class constraint

The cleaned parquet contains both pre-trip and post-trip fields. A model that consumes post-trip fields “cheats”: it predicts a quantity using information that would not exist at prediction time. We therefore partition every column into *knowable at booking* versus *post-trip*, and admit only the former. Excluded everywhere are the receipt components—tip\_amount, tolls\_amount, total\_amount, congestion\_surcharge, airport\_fee, improvement\_surcharge, extra—together with trip\_duration\_min and speed\_mph (unknown until the trip ends) and the dropoff timestamp. The tip model additionally excludes payment\_type; Section 3 explains why this is the single most consequential exclusion in the suite.

## 3 Exploratory Analysis

Three findings from exploratory analysis shaped every subsequent modeling choice.

**Temporal structure is joint but not lockstep.** Hour-of-day moves fare, tip rate, and demand together but along different curves. Overnight hours (00:00–04:00) carry the highest mean total (\$26–\$34) yet the lowest tip rate; tip rate peaks in the early-morning commuter window; pickup demand is bimodal with AM and PM rush peaks. This argues for treating pickup\_hour as a categorical feature so the tree can learn these non-monotone interactions directly through subset splits, rather than forcing a linear or ordinal hour effect.

**Cash silently zeros the tip target.** The TLC field tip\_amount is populated only for credit-card transactions; cash tips are never recorded. In the cleaned data, credit-card trips (29.5M) show a 95.6%

recorded-tip rate, while cash trips (6.1M) show essentially 0%. This is a property of the *measurement*, not of riders. It has a sharp methodological consequence: a model that includes `payment_type` is not predicting generosity, it is predicting whether the fare was paid by card. An interpretable logistic baseline that included `payment_type` reached AUC 0.916 and 96% accuracy—but almost all of that apparent skill is the payment-method proxy. Because payment method is chosen *after* booking, the production tip model must exclude it, and the prediction must be read as a joint quantity,  $P(\text{card}) \times P(\text{tip} \mid \text{card})$ .

**Borough geography dominates distance and demand.** Manhattan-to-airport pairs anchor the right tail of the fare distribution; same-borough trips drive the lowest distances and the highest tip rates. This motivated borough-centroid distance proxies (Section 4), which give the trees a finer geographic signal than the categorical borough label alone.

## 4 The Feature Contract

The contract is six numeric and ten categorical columns (Table 1). Three of the numeric columns and several flags are engineered along three pillars.

**Table 1.** The 16-column feature contract. Every column is observable at booking time; the listed fields are excluded as leakage. Categorical columns enter XGBoost as native `category` dtype (no one-hot expansion).

| Column                       | Description                                        |
|------------------------------|----------------------------------------------------|
| <i>Numeric (6)</i>           |                                                    |
| <code>trip_distance</code>   | meter distance, mi (routing estimate at inference) |
| <code>haversine_mi</code>    | great-circle dist., borough centroids              |
| <code>manhattan_mi</code>    | $L_1$ grid distance, borough centroids             |
| <code>passenger_count</code> | integer, 1–6                                       |
| <code>temp_f</code>          | daily mean temperature (°F)                        |
| <code>precip_in</code>       | daily precipitation (in)                           |
| <i>Categorical (10)</i>      |                                                    |
| <code>pickup_hour</code>     | hour of day, 0–23                                  |
| <code>pickup_dow</code>      | day of week, 0–6                                   |
| <code>is_weekend</code>      | Saturday/Sunday flag                               |
| <code>is_rainy</code>        | precipitation >0.1 in                              |
| <code>PU_borough</code>      | pickup borough (6)                                 |
| <code>DO_borough</code>      | dropoff borough (6)                                |
| <code>VendorID</code>        | TLC license vendor                                 |
| <code>RatecodeID</code>      | trip type (incl. JFK flat rate)                    |
| <code>is_us_holiday</code>   | 11 federal holidays                                |
| <code>is_nyc_event</code>    | Marathon, Pride, NYE, ...                          |

### Excluded (leakage / post-trip)

`tip_amount`, `tolls_amount`, `total_amount`,  
`congestion_surcharge`, `airport_fee`, `trip_duration_min`,  
`payment_type` (tip model only)

**Pillar 1: event-day flags.** `is_us_holiday` marks the eleven U.S. federal holidays; `is_nyc_event` marks a curated set of high-impact city days (NYC Pride, the US Open final, the NYC Marathon, the Thanksgiving Day Parade, and New Year’s Eve in Times Square). Each of the four predicted quantities behaves measurably differently on these dates, and borough-specific volume spikes are invisible to federal-holiday flags alone.

**Pillar 2: distance proxies.** Two geometric distances between borough centroids approximate the routing-API estimate a production system would have at request time. The haversine (great-circle) distance is

$$d_{\text{hav}} = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}}$$

where  $\phi$  is latitude and  $\lambda$  is longitude (both in radians),  $\Delta\phi$ ,  $\Delta\lambda$  are their pickup–dropoff differences, and  $R=3958.8$  mi is Earth’s radius. The Manhattan- $L_1$  proxy,  $|\Delta\phi| + |\Delta\lambda|$  converted to miles, is a coarse street-grid estimate. The gap between the two proxies, and between either proxy and the recorded meter distance, encodes route-complexity signal.

**Pillar 3: weather join.** Daily Central Park temperature and precipitation are joined on pickup date from the Open-Meteo historical archive [6], with `is_rainy` set for precipitation above 0.1 in. A seeded synthetic-climatology fallback keeps the join deterministic when the network is unavailable.

## 5 Modeling Methodology

### 5.1 One pipeline, four heads

The four models share their entire pipeline up to the objective: the same cleaned parquet, the same sanity filter (with task-specific bounds), the same three engineering pillars, the same stratified sampling, the same Optuna search, and the same cross-validation. Only the model *head* differs (Table 2).

## 5.2 Native categorical splits

We drop one-hot encoding entirely. One-hot expansion of `pickup_hour` (24), `pickup_dow` (7), the two boroughs (6×6), `VendorID`, and the flags inflates the design matrix to roughly fifty transformed columns. Instead, categorical columns enter XGBoost as the pandas `category` dtype with `enable_categorical=True`. The tree’s optimal-partition algorithm can then split on a *subset* of categories at a node—for example, `hour∈{7,8,9,17,18,19}` in a single split—which is strictly more expressive than a one-hot model’s single-value splits. The practical payoffs are lower memory, sixteen columns instead of fifty, and feature importances that read at the source-feature level with no post-hoc aggregation.

## 5.3 Sampling

Two stratified samples are drawn, both keyed on `pickup_hour×PU_borough` to preserve the temporal-spatial mix: a 1.5M-row *evaluation* sample for cross-validation and a 250K-row *tuning* sample for Optuna trials. The smaller tuning sample trades per-trial precision for more trials per wall-clock minute, which is the right trade because later trials carry more information than marginally tighter early ones.

## 5.4 Hyperparameter search

We use Optuna’s default Tree-structured Parzen Estimator (TPE) [3], [2]. Rather than search blindly, TPE models the configuration density separately for good and poor trials. Writing  $\theta$  for a hyperparameter configuration and  $y$  for its validation loss, TPE maintains  $\ell(\theta)=p(\theta | y<y)$  (*densities that produced good losses*) and  $g(\theta)=p(\theta | y\geq y)$  (the rest), for a quantile threshold  $y^*$ , and proposes the  $\theta$  maximizing the ratio

$$\theta^* = \arg \max_{\theta} \frac{\ell(\theta)}{g(\theta)}$$

In practice it explores early, then concentrates on promising regions. Each of 30 trials does one stratified 80/20 split on the tuning sample, fits with 50-round early stopping, and minimizes the task metric (RMSE for the regressions, log loss for tip, Poisson deviance for demand). The search space (Table 2) spans tree depth, learning rate, ensemble size, leaf regularization

(`min_child_weight`, `reg_alpha`, `reg_lambda`, `gamma`), and row/column subsampling.

**Table 2.** Per-target objectives (top) and the shared Optuna search space (bottom). The whole pipeline above the head is identical across targets.

| Target                                      | Objective                  | Tuning metric / CV         |
|---------------------------------------------|----------------------------|----------------------------|
| Fare                                        | reg:squarederror           | RMSE; 5-fold               |
| Duration                                    | reg:squarederror           | RMSE; 5-fold               |
| Tip                                         | binary:logistic            | log loss; 5-fold           |
| Demand                                      | count:poisson              | Poisson dev.; Group 5-fold |
| <i>Optuna search space (30 trials, TPE)</i> |                            |                            |
| <code>max_depth</code>                      | integer 4–12               |                            |
| <code>learning_rate</code>                  | log-uniform $10^{-2}$ –0.3 |                            |
| <code>n_estimators</code>                   | integer 200–1500           |                            |
| <code>min_child_weight</code>               | integer 1–20               |                            |
| <code>subsample</code>                      | uniform 0.6–1.0            |                            |
| <code>colsample_bytree</code>               | uniform 0.6–1.0            |                            |
| <code>reg_alpha</code>                      | log-uniform $10^{-3}$ –10  |                            |
| <code>reg_lambda</code>                     | log-uniform $10^{-3}$ –10  |                            |
| <code>gamma</code>                          | uniform 0–5                |                            |

## 5.5 The demand target needs a different model

Demand is the structurally different task: there is no per-trip “demand” column. We *derive* the target by counting pickups per (`PU_borough`, date, hour) bucket and joining that count back to every trip in the bucket, yielding 43,355 buckets with a heavily right-skewed count distribution (median 36, mean  $\approx 809$ , maximum 9598 pickups). Counts are non-negative integers with variance growing with the mean, so we use Poisson regression with a log link:

$$P(\text{count} = k) = \frac{e^{-\mu} \mu^k}{k!}, \quad \mu = \exp(\beta^\top \mathbf{x})$$

where  $\mu$  is the expected count for a bucket,  $\mathbf{x}$  is the feature vector,  $\beta$  is the learned coefficient vector, and  $k$  is an observed count. XGBoost applies the inverse link internally, so the deployment consumes a count, not a log-count.

## 5.6 Cross-validation, and why grouping matters for demand

The three trip-level models use ordinary 5-fold cross-validation on the evaluation sample, reporting mean  $\pm$  standard deviation across folds; the standard deviation is the honest noise floor on each metric. The demand model cannot use ordinary folds: every trip in a bucket shares the *same* target value, so a random fold would place rows from one bucket in both train and validation and leak the target almost perfectly. We instead use `GroupKFold` keyed on the bucket identifier, guaranteeing each bucket appears entirely within one fold. The same grouping is used for the held-out split. This is the single most important correctness move in the demand notebook.

## 5.7 Reproducibility

A single seed (42) propagates through every stochastic operation: the NumPy generator, `train_test_split`, `KFold` and `GroupKFold`, the Optuna TPE sampler, and each XGBoost `random_state`. Given the same input parquet and environment, reported numbers reproduce bit-exactly; a save/reload round-trip of each booster was verified to agree to within  $10^{-6}$  on a probe prediction.

## 6 Results

Table 3 consolidates the four models. All cross-validation figures are mean  $\pm$  standard deviation on the 1.5M-row stratified sample; held-out figures are on a disjoint 20% split.

**Table 3.** Consolidated results. CV columns report mean  $\pm$   $\sigma$  over folds (group-aware folds for demand). Tip is evaluated by calibration-oriented metrics; its AUC is reported for completeness, not as the selection criterion. Exact objective strings are listed in Table 2.

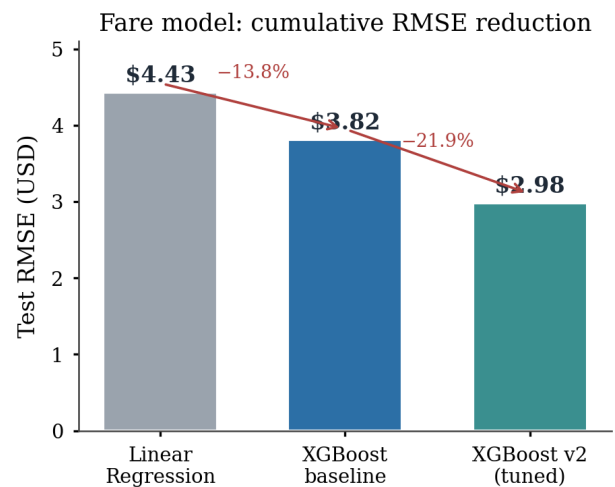
| Model    | Objective     | Primary (5-fold CV)         | Supporting (CV)               | Fit / calibration (CV)             | Held-out 20%                          |
|----------|---------------|-----------------------------|-------------------------------|------------------------------------|---------------------------------------|
| Fare     | squared error | RMSE \$2.985 $\pm$ 0.011    | MAE \$1.913 $\pm$ 0.002       | R <sup>2</sup> 0.9672 $\pm$ 0.0002 | RMSE \$2.979, R <sup>2</sup> 0.9672   |
| Duration | squared error | RMSE 5.414 $\pm$ 0.011 min  | MAE 3.605 $\pm$ 0.004 min     | R <sup>2</sup> 0.8236 $\pm$ 0.0011 | RMSE 5.410 min, R <sup>2</sup> 0.8232 |
| Tip      | logistic      | LogLoss 0.4954 $\pm$ 0.0011 | AUC 0.6220 $\pm$ 0.0018       | Brier 0.159; ECE 0.002             | LogLoss $\approx$ 0.495               |
| Demand   | Poisson       | MAE 241.2 $\pm$ 6.0 pickups | RMSE 359.2 $\pm$ 12.3 pickups | R <sup>2</sup> 0.9674 $\pm$ 0.0023 | MAE 248.7, R <sup>2</sup> 0.9627      |

### 6.1 Fare

The fare model is the suite’s strongest result and its clearest improvement story (Figure 2). An interpretable linear-regression baseline reaches \$4.43 RMSE. An untuned XGBoost baseline—one-hot encoded, hand-set hyperparameters, borough-level location, a single 80/20 split—reaches \$3.82 RMSE (R<sup>2</sup>=0.947). The tuned v2 model adds Optuna search, native categorical splits, the three feature pillars, and the `RatecodeID` feature, reaching \$2.98 RMSE and R<sup>2</sup>=0.967 under 5-fold CV, with the held-out split agreeing to the cent. That is a 21.9% relative RMSE reduction over the XGBoost baseline. A single change carried much of it: adding the rate-code feature alone moved fare RMSE from \$3.44 to \$2.98. The tuned configuration is moderate (depth 8, learning rate 0.034, 820 trees), not extreme, which is consistent with the tight cross-validation variance ( $\sigma$ =\$0.011).

The fare residuals are not uniform. The worst misses cluster in two predictable regimes the meter does not govern: JFK flat-rate trips, where a short recorded distance maps to a fixed \$70+ fare (one case: \$138

actual vs. \$36 predicted), and GPS-error micro-trips, where a near-zero distance pairs with a large fare. Tightening the v2 sanity bounds to the meter-driven regime (\$3–\$150, 0.3–50 mi) trims both tails.



**Figure 2.** Fare RMSE falls at each stage of refinement. Tuning, native categorical splits, feature engineering, and the rate-code feature together cut RMSE by 21.9% relative to the XGBoost baseline.



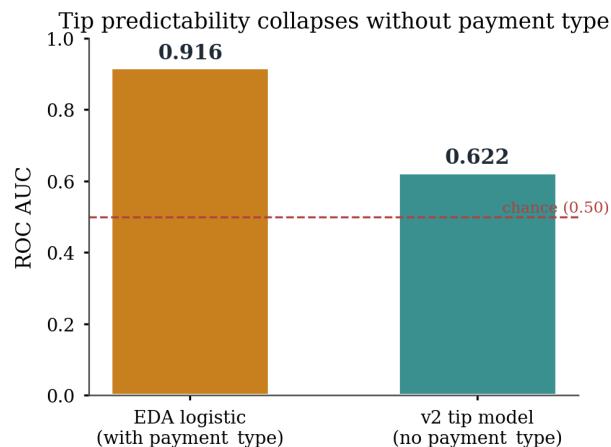
## 6.2 Duration

Duration shares the fare pipeline with a swapped target and a 1–120 min bound (the multi-hour tail is dominated by airport-queue and stuck-meter artifacts). It reaches 5.41 min RMSE, 3.60 min MAE, and  $R^2=0.824$ —honest to within about five minutes on a 14-minute mean trip. The lower  $R^2$  than fare is expected and not a defect: travel time carries irreducible variance (traffic, signal cycles, driver behavior) that route geometry and weather cannot capture, whereas a metered fare is a much more deterministic function of distance and rate code.

## 6.3 Tip: a calibrated near-null result

The tip model is where leakage discipline changes the conclusion. With `payment_type` correctly excluded, the model’s ranking ability collapses to AUC 0.62—barely above chance—and its accuracy (0.79) is essentially the base rate  $P(\text{tipped})=0.786$  (Figure 3). In other words, once we remove the payment-method proxy, *tip likelihood is nearly unpredictable from legitimate pre-trip features*. The apparent skill of the EDA logistic model (AUC 0.916) was almost entirely the payment signal.

This reframes the objective. Because the deployment displays a raw probability, the property that matters is calibration, not separability: “33%” must mean that 33% of historically similar trips were tipped. We therefore tune for log loss rather than AUC and report Brier score (0.159) and expected calibration error (ECE 0.002). The probability the user sees tracks the empirical positive rate to within a fifth of a percent. What residual signal exists is carried by `RatecodeID` (importance 0.38), which separates airport and negotiated-rate trips whose tipping norms differ from standard metered trips.



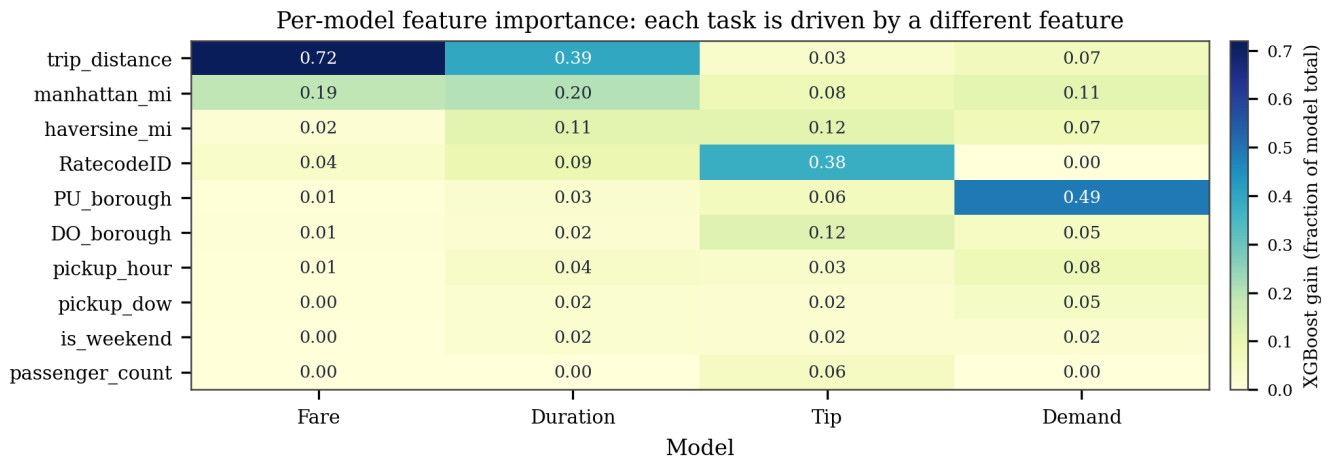
**Figure 3.** Removing `payment_type`—a post-booking signal that mostly encodes “paid by card”—collapses tip ranking from AUC 0.92 to 0.62. The honest pre-trip model is near chance, which is why we report calibration rather than AUC.

## 6.4 Demand

Under group-aware 5-fold CV, the Poisson demand model reaches MAE 241 pickups, RMSE 359 pickups, and  $R^2=0.967$ ; the held-out split gives MAE 249,  $R^2=0.963$ . The MAE must be read against the dynamic range: a Manhattan rush-hour bucket exceeds 1500 pickups while a Staten Island overnight bucket is in the single digits, so an absolute miss of a few hundred is small relative to the spread the model spans. As intended, the trip-level features (which carry no information about *other* trips in the same bucket) sink to near-zero importance, and the bucket-level signal dominates—bucket-level features account for 0.69 of total gain versus 0.31 for trip-level features.

## 6.5 Each task is driven by a different feature

The most compact summary of the suite is its feature-importance matrix (Figure 4). The four tasks, though they share one feature contract, are each dominated by a *different* variable: fare and duration by `trip_distance` and the distance proxies; tip by `RatecodeID`; demand by `PU_borough`. This is the empirical justification for the shared-contract design—one schema is rich enough that four heads can each find their own signal within it—and a useful sanity check, since each model relies on exactly the variable the domain says it should.



**Figure 4.** XGBoost gain (fraction of each model’s total) for the ten most-important contract features. Reading down the columns: the two regressions lean on distance, tip leans on rate code, and demand leans on pickup borough. One shared schema; four different stories.

## 7 Discussion: What the Results Led To

**One feature did most of the fare work.** Across the suite, the highest-leverage decision was not an algorithm but a feature. `RatecodeID` encodes trip type at pickup—standard meter, JFK flat rate, Newark, negotiated—and adding it alone moved fare RMSE from \$3.44 to \$2.98. It is also the dominant feature for tip and a strong one for duration. The lesson is mundane and important: when a target is governed by a small number of pricing regimes, a single categorical feature that names the regime can outweigh extensive hyperparameter tuning.

**Some targets are honestly hard, and saying so is the result.** The tip model is the suite’s most useful negative finding. Once the post-booking payment proxy is removed, pre-trip features cannot rank tippers (AUC 0.62). The defensible response is not to quietly reintroduce `payment_type` to inflate AUC; it is to change the objective. We optimize and report calibration, so the displayed probability is trustworthy even though its discriminative power is low. A well-calibrated “we mostly cannot tell” is a more honest deliverable than a high-AUC model leaking its answer.

**Validation design can dwarf model design.** For demand, the choice between ordinary and group-aware folds is not a tuning detail—it is the difference between a valid and an invalid number. Because every trip in a bucket shares a target, random folds leak that target and inflate the reported score by roughly an order of magnitude. The honest  $R^2=0.967$  comes only from group-keyed folds. In aggregated-target problems, how you split can matter more than what you fit.

**Honesty has an inference-time cost we name explicitly.** The fare model’s dominant feature, `trip_distance`, is a post-trip meter reading during training. At deployment it does not exist, so the demo estimates it as  $1.3\times$  the Manhattan- $L_1$  distance between borough centroids—a crude routing proxy. The model is therefore only as good as that estimate in production, and the principled fix (a real routing-API distance) is the first item of future work. We prefer to state this gap rather than let the training-time number imply a deployment-time guarantee.

## 8 Deployment

The four boosters are served through a single Streamlit application. A user selects pickup borough, dropoff borough, date, time, and passenger count; the app’s `build_feature_row` function emits exactly the 16-column contract—computing the distance proxies from centroids, joining weather (live forecast with a climatology fallback), setting holiday and event flags, and inferring `RatecodeID` from airport zone identifiers—and routes the row to every model through a typed wrapper that clamps each output to its training range (for example, a \$3 fare floor). Four metric tiles report fare, duration, tip probability, and demand. A “best time to leave” panel rebuilds the row at 15-minute steps across a three-hour horizon and surfaces the cheapest departure window; a debug expander displays the exact feature vector sent to the models, demonstrating that the contract is honored end to end. The contract is thus not merely a description—it is the executable integration spec, enforced by assertions in each notebook so that schema drift is caught before it reaches the demo.

## 9 Limitations and Future Work

Several limitations are direct consequences of the design choices above. *Distance*. Training on the recorded meter distance and approximating it at inference is the largest gap; replacing it with a pre-trip routing-API estimate would close the train/serve mismatch. *Spatial resolution*. Borough-level location is coarse; promoting PU\_borough/DO\_borough to the 263-zone PULocationID is feasible under native categorical handling without the one-hot blow-up,  $\approx 263$  usable zones of 265 IDs. *Temporal dynamics*. The demand model has no autoregressive memory; lagged features at  $t-1h$ ,  $t-24h$ , and  $t-168h$  would add strong signal but require a cache-backed inference path. *Tip decomposition*. A two-stage model that separately estimates  $P(\text{card})$  and  $P(\text{tip} \mid \text{card})$  would expose the measurement gap explicitly rather than folding it into one probability. *Surge readiness*. Quantile heads ( $P50/P90/P99$ ) would serve demand decisions better than a conditional mean, and a supply-side correction (realized trips = demand  $\times$  availability) would address the asymmetry on supply-constrained nights. *Engineering*. Lifting the duplicated feature-assembly code into one shared module, wrapping the boosters behind a service API, and adding versioned artifacts with drift monitoring are the production-readiness steps that follow the modeling work.

## 10 Conclusion

GridRunners shows that four heterogeneous taxi-prediction tasks can be unified behind a single leakage-disciplined feature contract, tuned and validated by one reproducible protocol, and served from one feature row. The quantitative headlines—\$2.98 fare RMSE, 5.41 min duration RMSE,  $R^2=0.967$  demand under group-aware validation—are real and reproducible. But the most transferable lessons are methodological: a single regime-naming feature can outweigh tuning; an honest near-null result, well calibrated, beats a leaky high-AUC one; and for aggregated targets, the validation split can matter more than the model. The contract is what makes these claims mutually consistent, and what turns four notebooks into a system.

## References

- [1] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.
- [2] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631.
- [3] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011.
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [5] New York City Taxi & Limousine Commission, “TLC trip record data,” NYC.gov. [Online]. Available: [nyc.gov/site/tlc/about/tlc-trip-record-data.page](https://nyc.gov/site/tlc/about/tlc-trip-record-data.page)
- [6] Open-Meteo, “Historical weather API.” [Online]. Available: [open-meteo.com](https://open-meteo.com)